

LAB 2: MULTILAYER PERCEPTRON (MLP)

OBJECTIVES:

- Implement the Backpropagation algorithm from scratch.
- Train an MLP to solve both linearly separable (AND) and non-linearly separable (XOR) logic gates.
- Visualize the decision boundaries learned by the network.

THEORY:

A Multilayer Perceptron (MLP) uses hidden layers to solve non-linear problems like XOR.

Backpropagation updates weights using the chain rule:

1. Forward Pass: $z = w * x + b$ and $a = \text{sigmoid}(z)$.
2. Loss Calculation: Measuring the difference between y_{pred} and y_{true} using Mean Squared Error.
3. Backward Pass:
 - Output Error (delta_out) = $(y_{\text{pred}} - y_{\text{true}}) * \text{sigmoid_derivative}(z_{\text{out}})$
 - Hidden Error (delta_hid) = $(\text{delta_out} * w_{\text{out_transpose}}) * \text{sigmoid_derivative}(z_{\text{hid}})$
4. Update Rule: $\text{weight} = \text{weight} - (\text{learning_rate} * \text{delta} * \text{input})$.

ALGORITHM:

1. Start
2. Initialize weights **w** to zeros and bias **b** to 0.
3. Set learning rate (**lr**) and maximum iterations.
4. Training Loop (for each epoch):
 - a. Set error_count = 0.
 - b. For each sample (x, y):
 - i. Compute linear output: $z = w * x + b$.
 - ii. Apply Step Function: **y_pred = 1 if z >= 0 else 0.**
 - iii. If **y_pred != y**: Update weights: **w = w + lr * (y - y_pred) * x** Update bias: **b = b + lr * (y - y_pred)** Increment error_count.
 - c. If error_count == 0, stop training (Converged).
5. End

SOURCE CODE:

```
# Cell 1: Setup and Functions
import numpy as np
import matplotlib.pyplot as plt

def sigmoid(x): return 1 / (1 + np.exp(-x))
def sigmoid_deriv(x): return x * (1 - x)

# Cell 2: Training and Visualization Logic
def run_mlp_lab(name, X, y):
    print(f"\n--- Training {name} Gate ---")
    wh = np.random.uniform(size=(2, 2))
    bh = np.random.uniform(size=(1, 2))
    wo = np.random.uniform(size=(2, 1))
    bo = np.random.uniform(size=(1, 1))

    lr, epochs = 0.5, 10000
    loss_history = []

    for i in range(epochs):
        # Forward
        h_act = sigmoid(np.dot(X, wh) + bh)
        o_act = sigmoid(np.dot(h_act, wo) + bo)

        # Loss
        loss = np.mean(np.square(y - o_act))
        loss_history.append(loss)

        # Backprop
        d_out = (o_act - y) * sigmoid_deriv(o_act)
        d_hid = d_out.dot(wo.T) * sigmoid_deriv(h_act)

        # Update
        wo -= h_act.T.dot(d_out) * lr
        bo -= np.sum(d_out, axis=0) * lr
        wh -= X.T.dot(d_hid) * lr
        bh -= np.sum(d_hid, axis=0) * lr

    # Print Results
    for j in range(len(X)):
        pred = 1 if o_act[j] > 0.5 else 0
        print(f"Input: {X[j]}, Target: {y[j]}, Pred: {pred}")
```

```

# Plotting
plt.figure(figsize=(10, 4))

# Loss Plot
plt.subplot(1, 2, 1)
plt.plot(loss_history)
plt.title(f'{name} Loss')
plt.xlabel("Epochs")

# Boundary Plot
plt.subplot(1, 2, 2)
x_range = np.linspace(-0.5, 1.5, 50)
xx, yy = np.meshgrid(x_range, x_range)
grid = np.c_[xx.ravel(), yy.ravel()]

h_grid = sigmoid(np.dot(grid, wh) + bh)
o_grid = sigmoid(np.dot(h_grid, wo) + bo).reshape(xx.shape)

plt.contourf(xx, yy, o_grid, alpha=0.3)
plt.scatter(X[:,0], X[:,1], c=y.flatten(), edgecolors='k')
plt.title(f'{name} Boundary')
plt.show()

# Cell 3: Execution
X = np.array([[0,0], [0,1], [1,0], [1,1]])
run_mlp_lab("AND", X, np.array([[0], [0], [0], [1]]))
run_mlp_lab("XOR", X, np.array([[0], [1], [1], [0]]))

```

OUTPUTS:

--- AND Gate ---

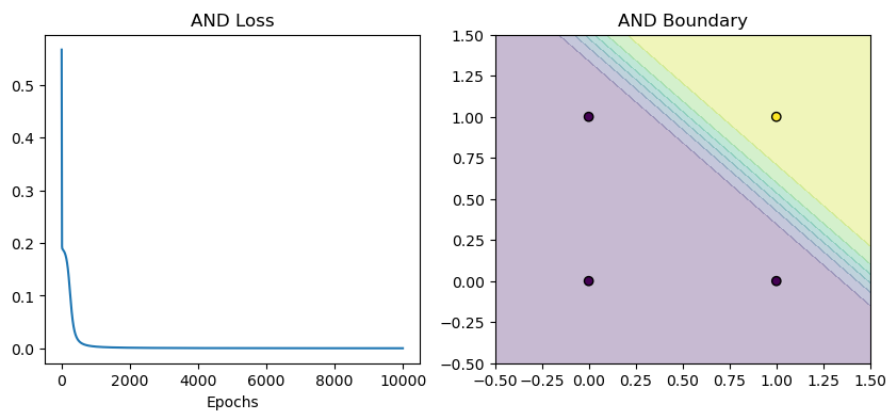
Converged after 4 epochs

Input: [0 0], Target: 0, Prediction: 0

Input: [0 1], Target: 0, Prediction: 0

Input: [1 0], Target: 0, Prediction: 0

Input: [1 1], Target: 1, Prediction: 1



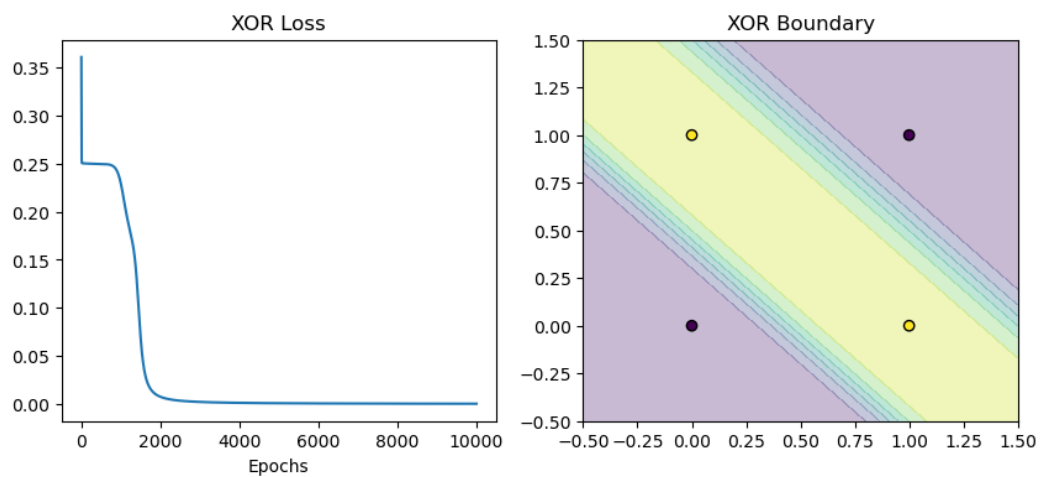
--- XOR Gate ---

Input: [0 0], Target: 0, Prediction: 1

Input: [0 1], Target: 1, Prediction: 1

Input: [1 0], Target: 1, Prediction: 0

Input: [1 1], Target: 0, Prediction: 0



CONCLUSION:

The perceptron converged for the AND gate but failed for XOR. This demonstrates that single-layer perceptrons are limited to linearly separable problems and cannot solve complex logic like XOR without additional layers or non-linear features.